



Наш блог

2025 (13) 2024 (10) 2023 (19) 2022 (10) 2021 (13) 2020 (10)

Перехват HTTPS-коммуникаций во Flutter: полный хардкорный режим с Frida

Время чтения ~27 мин.

Размещено Жаком Корце 16 апреля 2025 г.

Категории: [Flutter](#) , [Frida](#) , [Ghidra](#) , [Mobile](#) , [Pinning](#) , [Reversing](#) , [Ssl](#) , [Android](#) , [Certificate pinning](#)

tl;dr В этой записи блога я поделюсь идеями, которые я узнал, исследуя фреймворк Flutter и инструмент [reFlutter](#) . Я глубоко погружусь в архитектуру Flutter, некоторые из его внутренних механизмов и зависимостей, и, наконец, углублюсь в логику проверки SSL. Завершится запись исследованием того, что на самом деле делает инструмент reFlutter, и моими попытками воспроизвести то же поведение с помощью Frida.

Примечание: Если вы находитесь в затруднительном положении с мобильной оценкой, где приложение использует Flutter, инструмент reFlutter — отличный вариант. Эта запись в блоге не пропагандирует необходимость использования логики Frida. Это просто упражнение по выяснению того, может ли существовать эквивалент Frida.



Во время недавней оценки мы с коллегой столкнулись с проблемой на мобильной оценке только для планшета. В частности, мы пытались перехватить HTTPS-коммуникацию, но клиентское приложение не сотрудничало. В поисках решения я потратил некоторое время на загрузку своего Burp CA в системное хранилище планшета. Оказалось, это было довольно-таки хлопотно!

Даже с загруженным в системное хранилище Burp CA (в конце концов) мобильное приложение все еще не позволяло перехватывать свой HTTPS-трафик. Я пробовал использовать различные скрипты Frida, вручную маршрутизировать трафик через точку доступа Wi-Fi и некоторые модули Magisk, но ничего не работало. Затем я вернулся к архитектуре приложения и понял, что оно основано на фреймворке Flutter.

Это было не первое наше родео с Flutter. Другой коллега (привет, Джо) столкнулся и боролся с Flutter несколько лет назад. Основная проблема при пентестинге мобильных приложений с использованием Flutter заключается в том, что он использует собственные библиотеки для обработки связи и других мер безопасности. Следовательно, он не зависит от традиционной логики или библиотек Android (например, OkHttp3), которые мы привыкли видеть. Это также означает, что он не так легко поддается методам взлома, которые мы обычно применяем с Frida и Objection. Например, закрепление и проверка SSL выполняются на нескольких уровнях; Frida/Objection нелегко определит функции, ответственные за это.

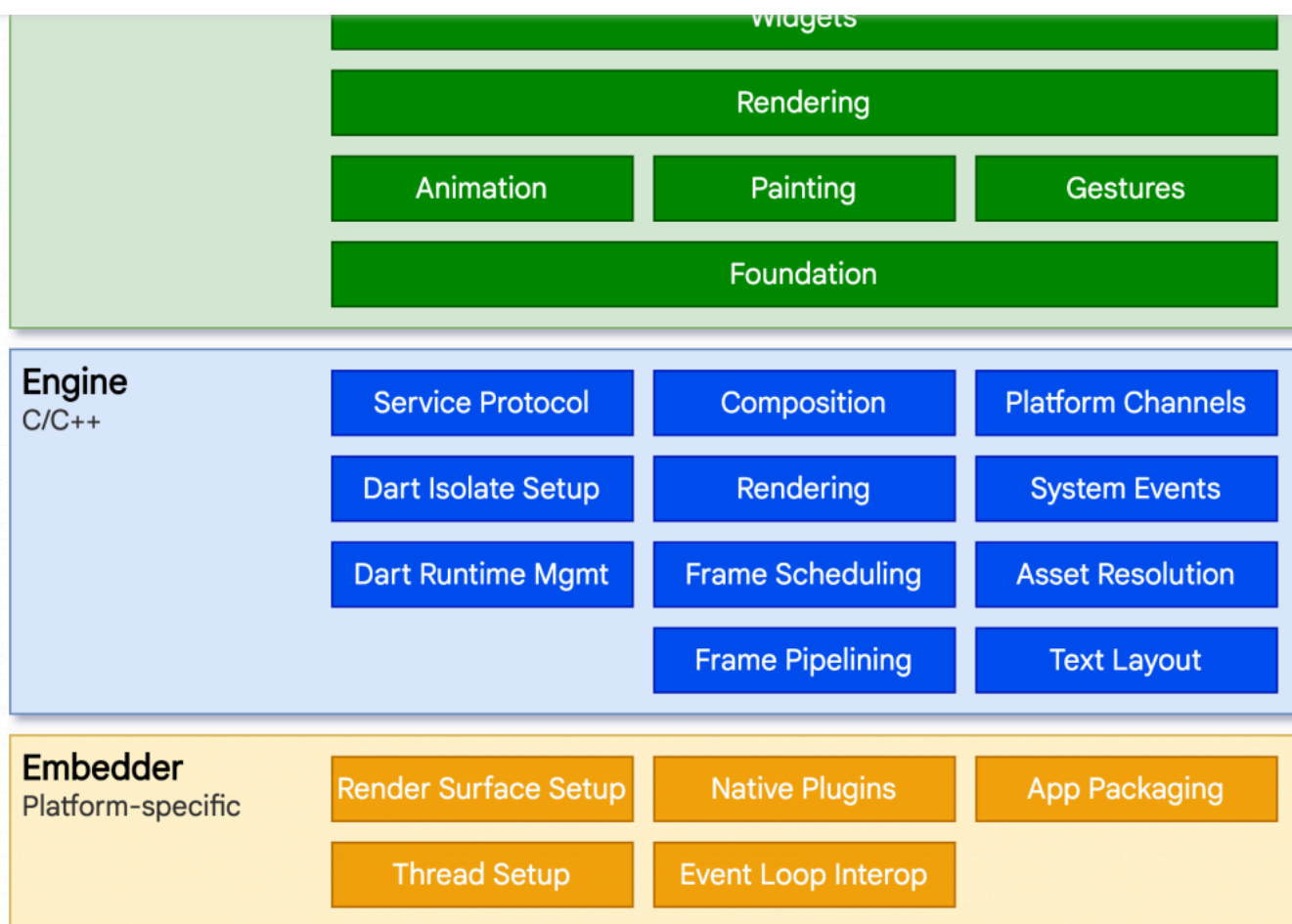
Из предыдущего опыта нашего коллеги мы знали об инструменте **reFlutter**. Мне было немного трудно запустить его, так как точная версия Flutter, используемая нашим текущим приложением, еще не была встроена в инструмент. После некоторого раздумья и нескольких контейнеров Docker я запустил его. Инструмент отлично справился с исправлением приложения и позволил перехватить его HTTPS-коммуникацию. Но это было похоже на магию, и я чувствовал себя скрипт-кидди. Поэтому мне стало интересно, что на самом деле делает



Что такое Flutter: введение в 101

Основная проблема для разработчиков приложений — это работа с различными мобильными экосистемами, Android и iOS. Flutter стремится абстрагировать различия от разработчиков, предоставляя единый унифицированный API, на который разработчики могут положиться.

По своей сути Flutter — это SDK. Этот SDK предоставляет элементы пользовательского интерфейса и другие общие элементы (например, HTTP/S-клиенты), которые за кулисами сопоставляются с собственными эквивалентами в сферах Android и iOS. SDK достигает этого посредством сочетания интеграций Dart и C/C++:

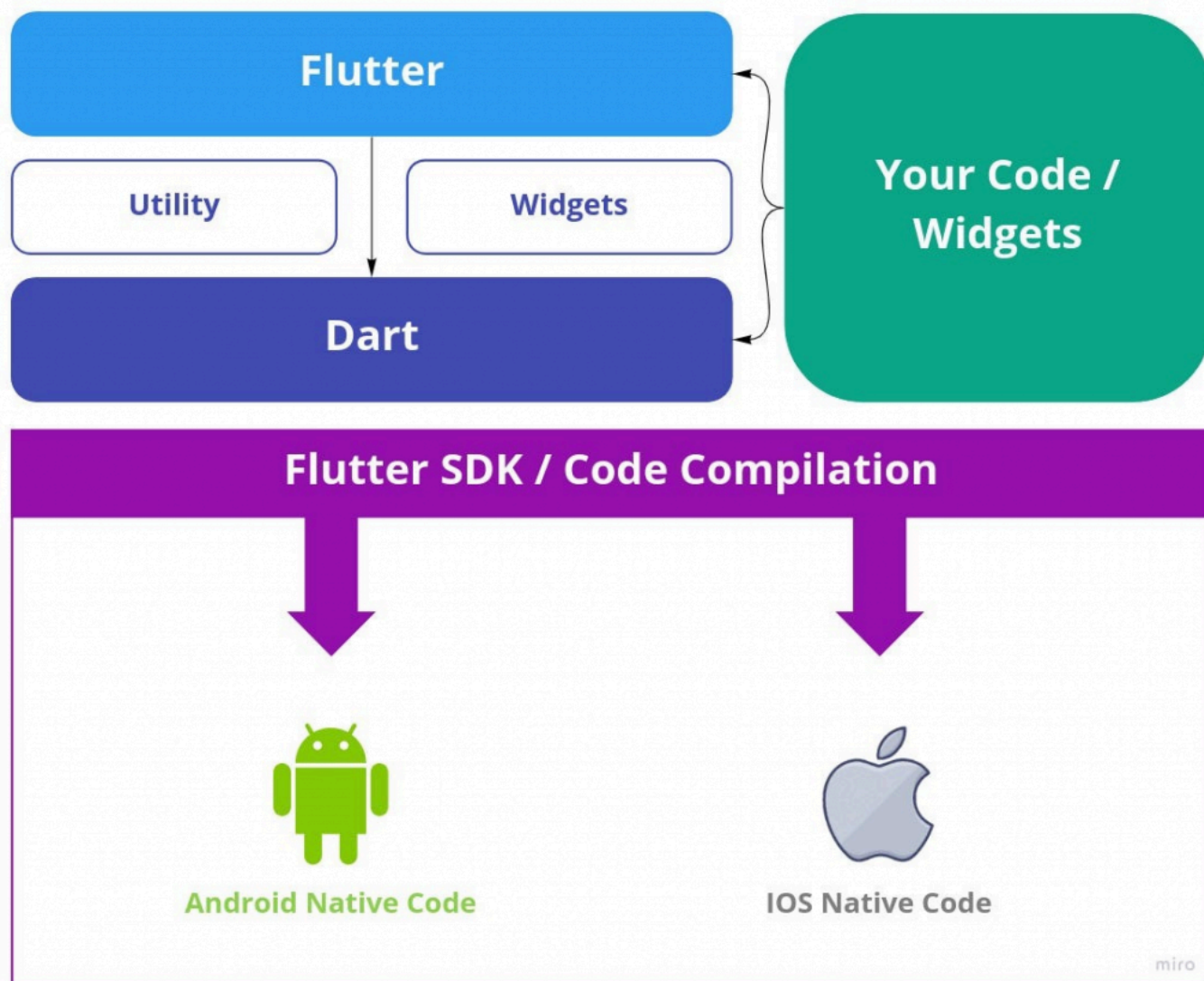


С точки зрения разработки Flutter разработчики пишут код, который взаимодействует с тем, что обычно называют **Framework**. Framework — это кроссплатформенный слой, написанный на Dart. Он включает в себя богатый набор платформ, макетов и базовых библиотек. Многие высокоуровневые функции, которые могут использовать разработчики, реализованы в виде пакетов, включая плагины платформы, которые могут: использовать камеру устройства, отображать WebView, участвовать в HTTP-коммуникации, отображать анимацию и выполнять другие функции.

Поскольку Flutter является кроссплатформенным, ему нужен унифицированный слой, который может транслироваться на Android и iOS. В Flutter этот унифицированный слой известен как **Engine**. Engine — это переносимая среда выполнения для хостинга приложений Flutter, которые содержат требуемый SDK для Android и iOS. Он в основном написан на C++ и предоставляет примитивы для поддержки всех приложений Flutter.



содержит Dart VM, которая преобразует эти инструкции в Java, которая в конечном итоге выполняется Android.



Однако есть один дополнительный аспект, который следует иметь в виду. Flutter — это не только фреймворк пользовательского интерфейса. Разработчик может расширить логику в движке с помощью собственного кода. Поэтому при компиляции приложения Flutter движок каждый раз создается заново — более известный как снимок приложения AOT (Ahead-of-Time). Снимок содержит машинный код как фреймворка Flutter, так и исходный код разработчика.

Учитывая вышесказанное, при рассмотрении структуры мобильного приложения Flutter вы обнаружите, что оно сильно отличается от нативных приложений Android и iOS.



```
├─ assets
│   └─ flutter_assets
│       └─ AssetManifest.json
├─ classes.dex — // Java (Dalvik Executable)
├─ kotlin — // kotlin Metadata
├─ lib
│   └─ arm64-v8a
│       ├── libapp.so — // Dart code (App AOT Snapshot)
│       └─ libflutter.so — // Flutter Engine (stripped version of Dart VM)
├─ META-INF
├─ res
└─ resources.arsc
```

Обратите внимание на вывод выше, что у нас есть `classes.dex` файл, а затем папка `lib`, содержащая несколько общих библиотек Flutter. В обычном приложении Android наше внимание будет сосредоточено на `classes.dex` файле. Используя **Jadx**, можно обратить инструкции Dalvik. Во Flutter это все еще возможно; но инструкции Dalvik делают немного больше, чем вызывают встроенные общие библиотеки Flutter.

Учитывая вышесказанное, наше внимание переключается на файлы общей библиотеки. Приложение Flutter обычно имеет **lib** папку; внутри нее вы найдете каталог, специфичный для архитектуры(-ий), для которой(-ых) было создано приложение. В случаях Android вы найдете ARM64 и/или ARM (т. е. `arm64-v8a`). Внутри этого каталога вы найдете два файла: **libapp.so** и **libflutter.so**.

Файл **libflutter.so** содержит требуемый функционал для использования ОС (сеть, файловая система и т. д.) и урезанную версию DartVM. С точки зрения оценки мобильных приложений этот файл становится основным фокусом; поскольку он содержит логику закрепления и проверки сертификата SSL, среди прочего!

The `libflutter.so` file does not contain execution logic and is incapable of loading the Dart source code itself. Instead, the **libapp.so** file is a wrapper (or, more aptly, a loader around it).

Diving Deeper into the Shared Libraries



While not technically required for assessments, it does help to know which version of Flutter was used by the mobile developer. At least since it might allow cross-referencing to specific functions or logic we might want to alter.

The libflutter.so and libapp.so files both contain an MD5 hash (known as the snapshot_hash) that corresponds with the build version. reFlutter contains a **script** that can be used on the libapp.so file to retrieve this hash:

```
python3 get_snapshot_hash.py libapp.so  
adb4292f3ec25074ca70abcd2d5c7251
```

In our application's case, the script returned the above MD5 hash. From this, we could now narrow down the exact Flutter version using reFlutter's **enginehash** file:

3.10.0	d44b5a94c976fbb65815374f61ab5392a220b084	aa64af18e7d086041ac127cc4bc50c5e
3.7.12	1a65d409c7a1438a34d21b60bf30a6fd5db59314	adb4292f3ec25074ca70abcd2d5c7251
3.7.0	b24591ed328a2d4ce6505819e2f5898a78132f7f	501ef5cbd64ca70b6b42672346af6a8a

The first column gives the Flutter version, while the second gives the engine hash. The engine hash is tied to a GitHub commit that contains all the dependencies and resources used to build that specific version of Flutter.

In our example, the commit and its dependencies would be found at:

<https://github.com/flutter/engine/blob/1a65d409c7a1438a34d21b60bf30a6fd5>

You might be wondering why the reference to the above commit is useful. To tamper with the Flutter logic, we need access to the specific Dart SDK on which the mobile application depends. After all, the Dart SDK is responsible for network communication – or in our case, SSL certificate verification.

Following the link above, we can discern the **exact Dart SDK commit** that the Flutter Engine depended on during the build process for Flutter 3.7.12:



'dart_revision': '5b8936496673a285bf8f37472119bc2afb38a295'

Once again, we can search for that revision in the Dart SDK GitHub repo:

<https://github.com/dart-lang/sdk/tree/5b8936496673a285bf8f37472119bc2afb38a295>

Looking at the [commit history](#), we find that the link refers to Dart SDK v2.19.6. The SDK can be rather complex, but the main area of interest is the `/runtime/bin` directory. Inside, you will find many C/C++ files containing the logic Flutter uses underneath.

If we wanted to tamper with the proxy configuration for the Flutter application, you will narrow your search to the `socket_android.cc` file:

https://github.com/dart-lang/sdk/blob/5b8936496673a285bf8f37472119bc2afb38a295/runtime/bin/socket_android.cc

Narrowing Down the X509 Verification Logic

So we can narrow down specific files and sections that we might want to tamper with, or alter, to change the behaviour of a Flutter application using the previous section's steps.

Unfortunately, when I went down the above route, I found references to X.509 in the Dart SDK, but I could not identify how the SSL certificate verification was done. Then I noticed some headers at the top of a few files:

```
#include <openssl/ssl.h>
#include <openssl/x509.h>
```

Online searches revealed that Flutter does not perform SSL certificate verification within the Dart SDK. Instead, it depends on a third-party SSL library. Returning to the [earlier Flutter dependency links](#), I narrowed it down to the BoringSSL library:



does not change that frequently. But one could use the commit hash above (originally from the Flutter dependency page) to narrow down the specific library version.

While scouring the BoringSSL library, I managed to identify that the SSL certificate verification logic resided in the `/ssl/ssl_x509.cc` file and the contained `ssl_crypto_x509_session_verify_cert_chain` function:

```
static bool ssl_crypto_x509_session_verify_cert_chain(SSL_SESSION *session,
                                                       SSL_HANDSHAKE *hs,
                                                       uint8_t *out_alert) {
    *out_alert = SSL_AD_INTERNAL_ERROR;
    STACK_OF(X509) *const cert_chain = session->x509_chain;
    if (cert_chain == nullptr || sk_X509_num(cert_chain) == 0) {
        return false;
    }

    SSL *const ssl = hs->ssl;
    SSL_CTX *ssl_ctx = ssl->ctx.get();
    X509_STORE *verify_store = ssl_ctx->cert_store;
    if (hs->config->cert->verify_store != nullptr) {
        verify_store = hs->config->cert->verify_store;
    }
}
```

The function returns a boolean that indicates whether the presented SSL certificate is valid or not. To bypass the verification logic, one needs to modify the return value to always be 'true'.

Note: The BoringSSL library within the context of Android Flutter applications does not utilise the device's CA stores. Instead, it uses its own store and, in some cases, may even depend on Google's own online verification routines.

However, the above logic resides several layers deep in the Flutter Engine file (**libflutter.so**). Since it is in native C/C++, there are no immediate references to it when using Frida or Objection. To further complicate things, the BoringSSL library and Dart SDK's symbols are stripped when the **libflutter.so** file gets created.

understanding at the time!

Patching with the reFlutter Tool

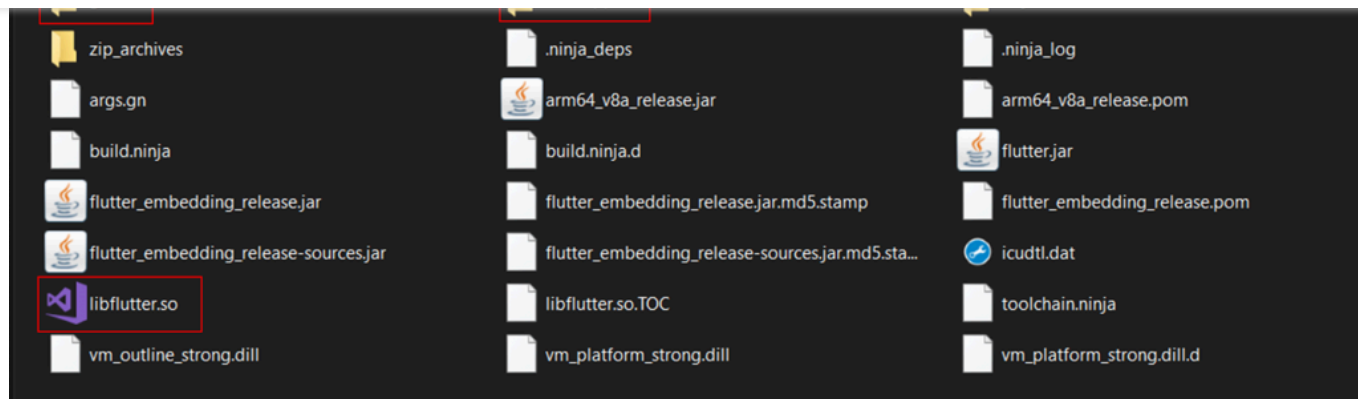
As we have seen in the previous sections, if one wants to modify the logic embedded within a Flutter application, it requires diving deeply into the dependency tree of the Flutter framework. This is a time-consuming exercise you would not want to repeat regularly.

The **reFlutter** tool was developed to simplify the process of tracking down specific security measures in the Dart SDK and patching them such that testing of Flutter-based applications is easier to perform.

The author (**Philip Nikiforov**) of the tool realised that due to the compression and stripping of symbols in the libflutter.so file; patching or modification on the file itself would be extremely difficult. One would basically have to reverse engineer and apply custom logic each time. As such, it would not be an easily repeatable process. Not to mention, each version of Flutter and the Dart SDK would change this process.

Instead, one can focus on the raw source code of the Engine being built into the libflutter.so file. Since Flutter and the Dart SDK is open-source, the build process to generate the file is publicly known (explained **here** and **here**). As such, one could start with the source code, modify it and then compile a custom version of the libflutter.so file – the caveat being you would need to know the exact Flutter version that the mobile application depends on, which we saw earlier is achievable.

Retracing the steps of the official Flutter build process, the reFlutter tool pulls down the Flutter SDK from GitHub and all its dependencies. It then creates a folder structure as follows:



Here, all the files needed to create a Flutter snapshot can be found: `artifacts`, `flutter_patched_sdk_product`, and `gen_snapshot`. The `lib.stripped` folder contains the `libflutter.so` engine, which will be copied into a folder with `libapp.so` during the compilation of the APK.

Inside the `lib.stripped` folder, you will find the source code for all the third-party dependencies. For example:

```
src/third_party/dart/runtime/bin/socket.cc <--- Proxy cap
src/third_party/boringssl/src/ssl/ssl_x509.cc <--- SSL c
```

To patch the BoringSSL certificate verification logic, the reFlutter tool uses regular Python text replacement to adjust the `ssl_x509.cc` file such that the verification logic is bypassed. The patching logic can be seen here:

<https://github.com/Impact-l/reFlutter/blob/main/src/init.py#L325>

Tracing the replacement logic, the earlier `ssl_crypto_x509_session_verify_cert_chain` function would be adjusted as follows:



```
+ return true;  
*out_alert = SSL_AD_INTERNAL_ERROR;  
STACK_OF(X509) *const cert_chain = session->x509_chain;  
if (cert_chain == nullptr || sk_X509_num(cert_chain) == 0) {  
    return false;  
}
```

Since the Flutter framework does not respect the Android device's proxy settings, the reFlutter tool also holds logic to adjust the Dart SDK's Socket creation logic (`socket.cc`) to force a proxy:

<https://github.com/Impact-I/reFlutter/blob/main/src/init.py#L322>

In the above case, the `socket.cc` file would be adjusted to include a hardcoded proxy IP address and port:

```
void FUNCTION_NAME(Socket_CreateConnect)(Dart_NativeArguments args) {  
    RawAddr addr;  
    SocketAddress::GetSockAddr(Dart_GetNativeArgument(args, 1), &addr);  
    Dart_Handle port_arg = Dart_GetNativeArgument(args, 2);  
    int64_t port = DartUtils::GetInt64ValueCheckRange(port_arg, 0, 65535);  
  
    + Syslog::PrintErr("ref: %s", inet_ntoa(addr.in.sin_addr));  
    + port=8083;  
    + addr.addr.sa_family=AF_INET;  
    + addr.in.sin_family=AF_INET;  
    + inet_aton("192.168.133.104", &addr.in.sin_addr);  
  
    SocketAddress::SetAddrPort(&addr, static_cast<intptr_t>(port));  
    if (addr.addr.sa_family == AF_INET6) {  
        Dart_Handle scope_id_arg = Dart_GetNativeArgument(args, 3);  
        int64_t scope_id =  
            DartUtils::GetInt64ValueCheckRange(scope_id_arg, 0, 65535);  
        SocketAddress::SetAddrScope(&addr, scope_id);  
    }  
    intptr_t socket = Socket::CreateConnect(addr);  
    OSError error;
```

Once the patching is completed, the `libflutter.so` is built from the adjusted source code. It can then be embedded into the original mobile application. After which, one can use an intercepting proxy, load a custom CA onto the mobile device and proxy network traffic.



using the reFlutter tool, it first checks whether a pre-patched file already exists for the targeted APK or IPA. If so, then it just downloads the necessary file. If not, you would have to manually perform the above steps – a Dockerfile helps with this process!

Going Hardcore: The Search for a Frida Equivalent

The patching that the reFlutter tool performs is done at source code level. It works fantastically well and gets the job done! But, I started wondering while helping my fellow colleague; whether one could achieve similar results using Frida. After all, it did not seem like an impossible task to achieve. At least, not for the certificate verification logic. Changing a boolean return type is simple – most of the time!

My search started by investigating the classes and class methods Frida and Objection could pick up at runtime within the target mobile application:

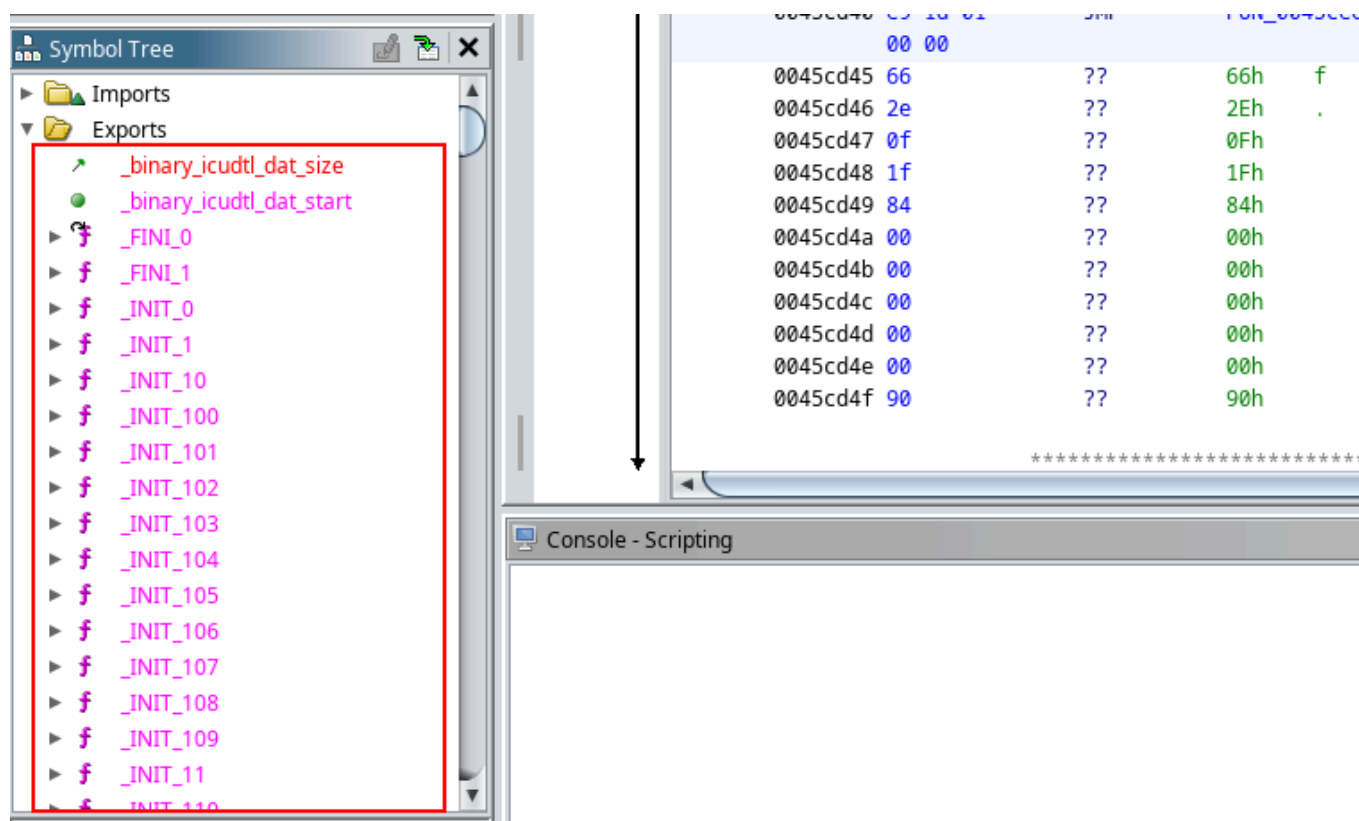
```
com.org.package on (google: 12) [usb] # android hooking 1
...
[Landroid.animation.Keyframe$FloatKeyframe;
[Landroid.animation.PropertyValuesHolder;
[Landroid.app.AppOpsManager$RestrictionBypass;
[Landroid.app.LoaderManagerImpl;
[Landroid.app.VoiceInteractor$request;
[Landroid.app.admin.PasswordMetrics$ComplexityBucket;
[Landroid.app.appsearch.AppSearchSchema$PropertyConfig;
...
```

I immediately encountered problems tracking down the classes from the libflutter.so file. Objection's 'hooking list classes' functionality only shows Java classes and not native C++ classes. As such, it would not show the classes from the libflutter.so file.

The second problem was even more complicated. One could possibly still hook the functions in the libflutter.so file provided that it exports those



Orange
Cyberdefense



Chatting with [@leonjza](#) (the creator of Objection), he mentioned that it might be fruitful to start by seeing whether I could locate the `libflutter.so` module in memory while the mobile application was running. Since that will possibly give entry to the functions hosted inside it.

I fired up Frida and attached it to the running application. Then used `Process.enumerateModules()` to see whether the `libflutter.so` file was loaded:

```
[Android Emulator 5554::com.org.package ]-> var modules =
[Android Emulator 5554::com.org.package ]-> modules
[
...
  {
    "base": "0x795181600000",
    "name": "libflutter.so",
    "path": "/data/app/~~QXyGrDRGt5-Lj2Jd_tmopw==/cor
    "size": 11165696
  },
```



Orange
Cyberdefense

As seen above, the libflutter.so file was indeed loaded into the running process. At a minimum, that allows us to get a handle on the module and inspect it:

```
[Android Emulator 5554::com.org.package ]-> var flutter =  
[Android Emulator 5554::com.org.package ]-> flutter  
{  
  "base": "0x795181600000",  
  "name": "libflutter.so",  
  "path": "/data/app/~~QXyGrDRGt5-Lj2Jd_tmopw==/com.org.  
  "size": 11165696  
}
```

Using the handle above, I started to explore the imports and exports of the module but quickly realised that it did not hold much success – as Ghidra confirmed as well. For example, the exports just showed this:

```
[Android Emulator 5554::com.org.package ]-> var exports =  
[Android Emulator 5554::com.org.package ]-> exports  
[  
  {  
    "address": "0x79518196cd99",  
    "name": "JNI_OnLoad",  
    "type": "function"  
  }  
]
```

Out of ideas, I revisited my earlier Google searches. In doing so, I came across references to two public blog posts from 2019 and 2020 where Jeroen Beckers had bypassed the SSL certificate pinning in Flutter using Frida. Reviewing those blog posts, I realised that Jeroen was using memory pattern matching to grab the specific functions (i.e. `ssl_verify_result`) from within the libflutter.so module:



Orange
Cyberdefense

```
var res = Memory.scan(m.base, m.size, pattern, {  
  onMatch: function(address, size){  
    console.log('[+] ssl_verify_result found at: ' + address  
  
    // Add 0x01 because it's a THUMB function  
    // Otherwise, we would get 'Error: unable to intercept f  
    hook_ssl_verify_result(address.add(0x01));  
  
  },  
  onError: function(reason){
```

I retraced the steps he published but had no success replicating the result. Naturally, the Flutter and Dart SDKs had changed since 2019, which meant the function(s) he was hooking did not exist anymore – or were not the same. That said, the blog posts did narrow down which function(s) might be useful.

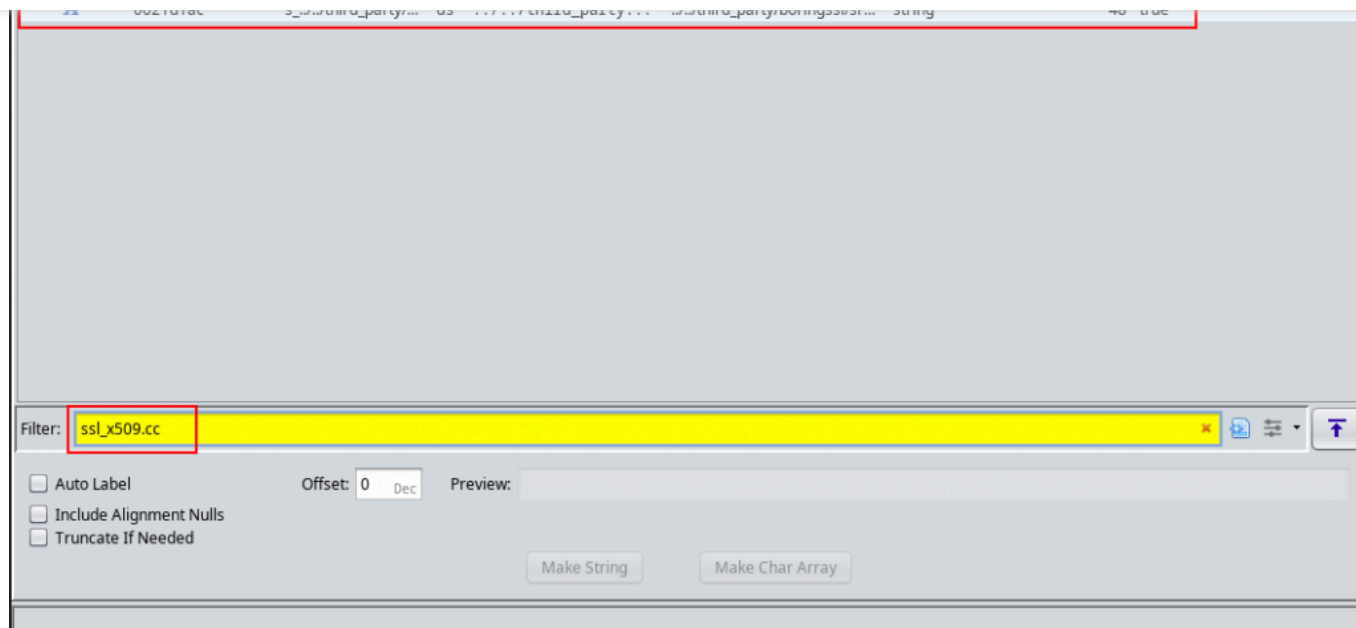
Continuing my research online, I noticed that Jeroen had released an update in 2022:

<https://blog.nviso.eu/2022/08/18/intercept-flutter-traffic-on-ios-and-android-http-https-dio-pinning/>

Although not exactly the same as our current application, he mentioned briefly in the update that Flutter had shifted some logic. In the latest iterations, the target function to bypass the SSL certificate verification logic is `ssl_crypto_x509_session_verify_cert_chain`.

Righto, so it is the same one reFlutter uses and the one we identified earlier. I had the target function at this stage, but I still needed to track down a way of hooking it in memory using Frida. To achieve this, I had to find the assembly instructions that denoted the start of the function – similar to Jeroen's script above.

Back in Ghidra, I searched for string reference to the `ssl_x509.cc` file:



From the search, I managed to learn that the libflutter.so file (at least, the one present in our current application) held nine references to the file:



These are cross-references to anything Flutter and the Dart SDK depends on within the BoringSSL library. As such, within the nine references, I had to track down which was the `session_verify_cert_chain` function.

Unfortunately, identifying the function turned into quite the exercise. Remember the libflutter.so file is stripped, which means no symbols are available. For example, this is the output Ghidra showed when navigating to a cross-reference of the `ssl_x509.cc` file:



```
long *plVar3;  
long lVar4;  
char cVar5;  
undefined8 uVar6;  
long **pplVar7;  
undefined8 uVar8;  
undefined8 uStack_60;  
long local_50;  
long local_48;  
long local_40;  
long local_38;  
undefined4 uStack_30;  
undefined4 uStack_2c;
```

```
if (param_2 == 0) {  
    FUN_007040e8(0x10, 0x43, ".../third_party/boringssl/src/ssl/ssl_x509.cc", 0x2e0);  
    return 0;  
}  
FUN_0075912a(&local_40);  
lVar4 = local_40;
```

At first, I was lost in how one would make sense of these cross-references. My search online continued, but I mainly found other blog posts referencing Jeroen's research. That is, until I managed to track down this Vietnamese post:

<https://sec.vnpt.vn/2022/04/frida-hooking-va-bypass-ssl-pinning-cho-flutter-application/>

In the blog post, the author had the following note:

“

The definition of the `session_verify_cert_chain` function is in the file `ssl_x509.cc`.

Okay, now the goal is to change the above function's return to true. And also note that the above function has 3 arguments passed (will help us find the correct function when decompiled with Ghidra).

”



Orange
Cyberdefense

receive three arguments:

```
static bool ssl_crypto_x509_session_verify_cert_chain(SSL_SESSION *session,
                                                       SSL_HANDSHAKE *hs,
                                                       uint8_t *out_alert) {
    *out_alert = SSL_AD_INTERNAL_ERROR;
    STACK_OF(X509) *const cert_chain = session->x509_chain;
    if (cert_chain == nullptr || sk_X509_num(cert_chain) == 0) {
        return false;
    }
}
```

As such, I could narrow my search to a cross-reference having a signature with three arguments. Fortunately, it turns out that only one of the nine cross-references satisfied this requirement:

```
ulong FUN_00759aea(long param_1, long *param_2, undefined *param_3)
{
    long *p1Var1;
    long lVar2;
    long lVar3;
    undefined8 uVar4;
    undefined uVar5;
    int iVar6;
    long lVar7;
    long lVar8;
    long lVar9;
    ulong uVar10;
    char *pcVar11;
    undefined8 uVar12;
    long lVar13;
    long local_38;

    *param_3 = 0x50;
    p1Var1 = *(long **)(param_1 + 0xa8);
    if ((p1Var1 == (long *)0x0) || (*p1Var1 == 0)) {
        uVar10 = 0;
        goto LAB_00759cd3;
    }
    lVar8 = *param_2;
    lVar2 = *(long *)(lVar8 + 0x68);
    lVar13 = *(long *)((long *)lVar8 + 0x30) + 0x110;
}
```

I do not know how this code maps back to the original logic. Perhaps it comes down to Ghidra's reversing of the library, but without the Vietnamese blog post, I would never have known this was the function.



00759aea	55	PUSH	RBP
00759aeb	41 57	PUSH	R15
00759aec	41 56	PUSH	R14
00759aef	41 55	PUSH	R13
00759af1	41 54	PUSH	R12
00759af3	53	PUSH	RBX
00759af4	48 83 ec 38	SUB	RSP, 0x38
00759af8	c6 02 50	MOV	byte ptr [RDX], 0x50
00759afb	48 8b af	MOV	RBP, qword ptr [RDI + 0xa8]
	a8 00 00 00		
00759b02	48 85 ed	TEST	RBP, RBP
00759b05	74 70	JZ	LAB_00759b77
00759b07	48 83 7d	CMP	qword ptr [RBP], 0x0
	00 00		
00759b0c	74 69	JZ	LAB_00759b77
00759b0e	4c 8b 2e	MOV	R13, qword ptr [RSI]
00759b11	48 8b 46 08	MOV	RAX, qword ptr [RSI + 0x8]
00759b15	4d 8b 4d 30	MOV	R9, qword ptr [R13 + 0x30]
00759b19	4d 8b 55 68	MOV	R10, qword ptr [R13 + 0x68]
00759b1c	48 8b 40 20	MOV	RAX, qword ptr [RAX + 0x20]
00759b21	48 8b 4d 08	MOV	RCX, qword ptr [RBP + 0x8]
00759b25	4d 8b 81	MOV	R8, qword ptr [R9 + 0x110]
	10 01 00 00		

Note: The assembly shown above is taken from the x86_64 libflutter.so file; since I was using an Android Studio emulator.

With the assembly instructions in hand, I revisited Jeroen's blog posts and took inspiration from the Frida scripts he had previously written to write my own:

```
function hook_ssl_verify_result(address) {
  Interceptor.attach(address, {
    onEnter: function (args) {
      console.log('Disabling SSL validation');
    },
    onLeave: function (retval) {
      console.log('Retval: ' + retval);
      retval.replace(0x1);
    },
  });
}
```



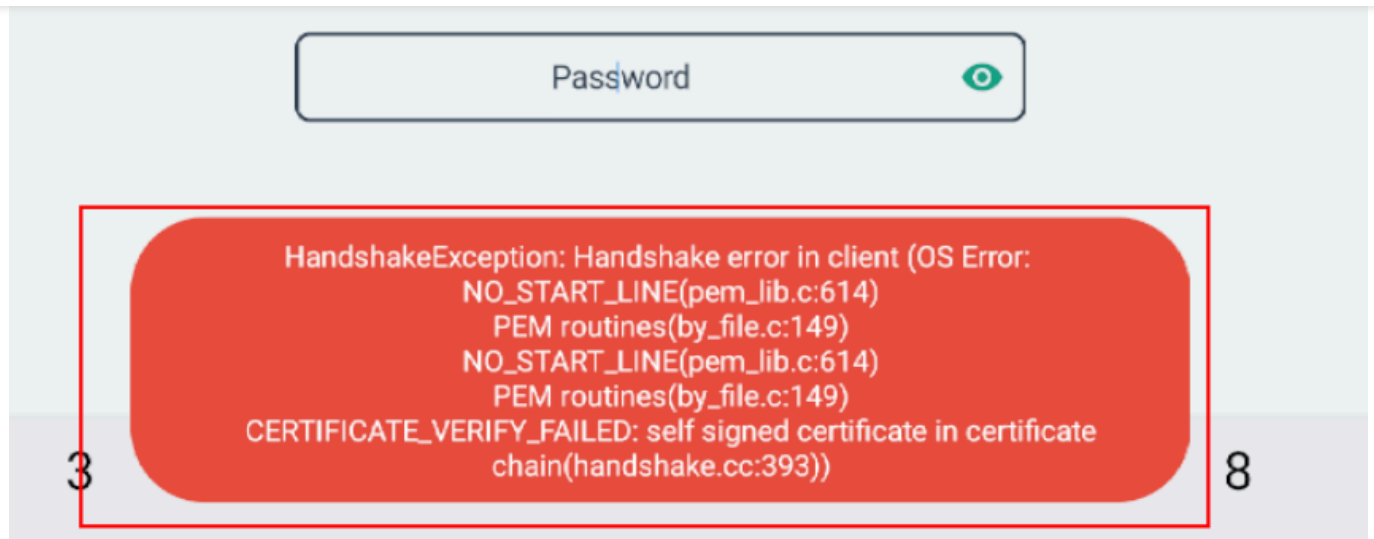
Orange
Cyberdefense

```
'55 41 57 41 56 41 55 41 54 53 48 83 ec 38 c6 02 50 48 8b af a8 00 00 00
var res = Memory.scan(m.base, m.size, pattern, {
  onMatch: function (address, size) {
    console.log('[+] ssl_verify_result found at: ' + address);

    hook_ssl_verify_result(address.add(0x00));
  },
  onError: function (reason) {
    console.log('[!] There was an error scanning memory');
  },
  onComplete: function () {
    console.log('All done');
  },
});
}
setTimeout(disablePinning, 1000);
```

Showcasing the Frida Script

To close out the blog post, I will first demonstrate the failure when an intercepting proxy (**Burp Suite**) is introduced to proxy the Flutter application without any bypass:



The screenshot above shows that the BoringSSL library rejects the SSL handshake since the endpoint's certificate is now self-signed. Remember that this occurs even when the Burp CA is loaded in the system CA store!

The issue above stems from the fact that the `session_verify_cert_chain` function returns an invalid response, which then causes the SSL handshake to be aborted.

```
break;
}
} else {
    ret = ssl->ctx->x509_method->session_verify_cert_chain(
        hs->new_session.get(), hs, &alert)
        ? ssl_verify_ok
        : ssl_verify_invalid;
}

if (ret == ssl_verify_invalid) {
    OPENSSL_PUT_ERROR(SSL, SSL_R_CERTIFICATE_VERIFY_FAILED);
    ssl_send_alert(ssl, SSL3_AL_FATAL, alert);
}

// Emulate OpenSSL's client OCSP callback. OpenSSL verifies certificates
// before it receives the OCSP, so it needs a second callback for OCSP.
```

If the Frida script above works, we can change that response allowing the SSL handshake to complete. At that point, we can intercept the network traffic and go on our way!



Orange
Cyberdefense

```
frida -U -l bypass.js -f com.org.package
```

Shortly after the application spawns, the Frida script hooks the intended function based on the assembly instructions in memory:

```

└─[23-08-01|15:47:04]─o frida -U -l bypass.js -f com.
  / _ |   Frida 15.2.2 - A world-class dynamic instrumentation toolkit
 | (| |   Commands:
 > _ |     help      -> Displays the help system
 /_/ |_    object?   -> Display information about 'object'
 . . .     exit/quit -> Exit
 . . .
 . . .     More info at https://frida.re/docs/home/
 . . .
 . . .     Connected to Android Emulator 5554 (id=emulator-5554)
Spawning `com.
Script loaded successfully
Spawned `com. Use %resume to let the main thread start executing!
[Android Emulator 5554::com. ]-> %resume
[Android Emulator 5554::com. ]-> Script loaded successfully
[+] ssl_verify_result found at: 0x789d13961aea
All done

```

Then the moment of truth! Can we raise communication from the Flutter application while proxying through Burp Suite? Absolutely, yes!

Request	Response
Pretty Raw Hex <pre> 1 POST /sync/authenticate HTTP/2 2 Host: 3 User-Agent: Dart/2.19 (dart:io) 4 X-Device-Name: google%20-%20sdk_gphone64_x86_64 5 Accept-Encoding: gzip, deflate 6 Content-Length: 81 7 X-Device-Id: null 8 Content-Type: application/x-www-form-urlencoded; charset=utf-8 9 10 grant_type=password&username=darryn.cull%40orange cyberdefense.com&password= </pre>	

Back in the Frida console, we notice that the hooked function originally would have returned a `0x1` response, but our script modified it such that it returned `0x0` instead:



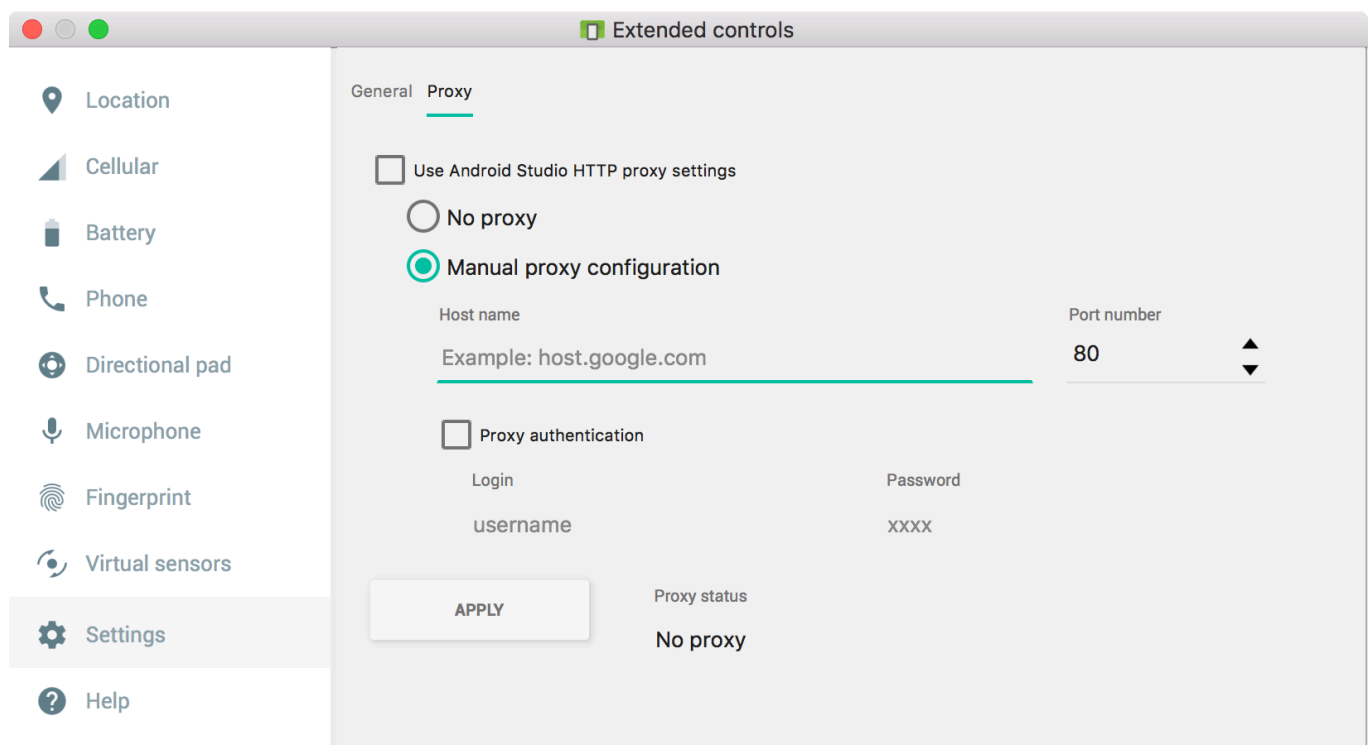
```
Disabling SSL validation  
Retval: 0x0
```

As such, we bypassed the SSL certificate verification step, and the SSL handshake was completed with Burp Suite without interference. Naturally, at this point, the mobile assessment would start by focusing on the network traffic and any vulnerabilities it may contain.

Sidebar: Proxying the Traffic

If you have a keen eye, you might recall earlier in the blog post, I said Flutter is not proxy aware. So you might be asking yourself, how did I intercept the traffic above?

Since I am using an Android Studio emulator, I control networking at the hardware/simulation level. Within Android Studio, you can set [a global proxy](#) for an emulated device by navigating to the device's Settings -> Proxy:



Note: More recent versions of Android Studio have changed where this setting is located. However, it is still possible to configure.



Orange
Cyberdefense

As an aside, if you are using a regular Android device, you could achieve the same result by setting up a custom `/etc/hosts` file on the device or using a custom Wi-Fi access point + DNS server and pointing specific hostnames to your Burp Suite instance's IP address.

Conclusion & Summary

You've made it to the end of this blog post. An achievement in itself!

In this blog post, I shared insights into how the Flutter framework is structured and its different elements. We learned that underneath the hood, it employs the Dart SDK alongside several other dependencies (i.e. BoringSSL).

Through exploration of the reFlutter tool, we learned that one can patch the raw source code of the Flutter Engine before it is compiled and embedded into an APK or IPA. To this extent, the reFlutter tool does much of the heavy lifting, making the patching process significantly easier. More so, since the associated GitHub repository contains several pre-patched `libflutter.so` files for Android and iOS.

With a clear understanding of the inner workings of the reFlutter tool, attention shifted to a Frida equivalent. Using several public blog posts, the `libflutter.so` file and stumbling around Ghidra, I wrote a Frida script to hook the `session_verify_cert_chain` function within the BoringSSL library and bypass Flutter's SSL certificate verification logic.

To be clear, the Frida script only caters for the `x86_64` architecture and the `libflutter.so` file found within an application I helped a colleague with. That said, this blog post's steps and the Frida script could serve as a template for the steps to be repeated for ARM and ARM64; if somebody finds themselves dealing with Flutter on a real device soon.



Orange
Cyberdefense

strange beast to encounter and requires quite a bit of insight into Dart, C/C++ and reverse engineering with IDA or Ghidra.

Until next time, stay curious and happy hacking!

References

- <https://swarm.ptsecurity.com/fork-bomb-for-flutter/>
- <https://blog.nviso.eu/2019/08/13/intercepting-traffic-from-android-flutter-applications/>
- <https://blog.nviso.eu/2020/06/12/intercepting-flutter-traffic-on-ios/>
- <https://blog.nviso.eu/2022/08/18/intercept-flutter-traffic-on-ios-and-android-http-https-dio-pinning/>
- <https://github.com/Impact-l/reFlutter/>
- <https://docs.flutter.dev/>
- <https://github.com/frida/frida/issues/434>
- <https://github.com/frida/frida/issues/2265>
- <https://frida.re/docs/javascript-api/>
- <https://raphaeldenipotti.medium.com/bypassing-ssl-pinning-on-android-flutter-apps-with-ghidra-77b6e86b9476>
- <https://www.horangi.com/blog/a-pentesting-guide-to-intercepting-traffic-from-flutter-apps>
- https://github.com/google/boringssl/blob/master/ssl/ssl_x509.cc
- <https://sec.vnpt.vn/2022/04/frida-hooking-va-bypass-ssl-pinning-cho-flutter-aplication/>
- <https://github.com/horangi-cyops/flutter-ssl-pinning-bypass/blob/main/flutter-bypass-sslpinning.js>
- <https://blog.nviso.eu/2020/11/19/proxying-android-app-traffic-common-issues-checklist>
- <https://medium.com/hackernoon/intercept-https-traffic-on-a-android-emulator-46023f17f6b3>



Orange
Cyberdefense

sensepost@orange cyberdefense.com

Пожалуйста, выберите тип вашего запроса, и мы свяжемся с вами как можно скорее.

Общий

Имя

Адрес электронной почты

Контактный номер

Ваше сообщение



Свяжитесь с нами

Нажимая «Связаться», вы соглашаетесь с Условиями обслуживания Orange Cyberdefense.

Претория (Главный офис)

+27 (0)12 460 0880

Park Lane West, 197 Amarand Ave, Уотерклүф-Глен, Претория, Южная Африка

Лондон (Головной офис)

+44 (0)2070 781 360

SensePost, 250 Waterloo Road, SE1 8RD, Лондон, Соединенное Королевство



Orange
Cyberdefense

183 Albion Springs Corner Main Road &, Albion Springs Cl., Рондебош,
Кейптаун, Южная Африка

© Orange Cyberdefense 2025